Atelier DORA Accelerate pied au plancher - Aide pour le facilitateur

Présentation de l'aide **3**
Présentation et préparation de l'atelier **3**
Présentation de l'atelier 3
Organisation de l'atelier 4
Préparation de l'atelier en présentiel avec utilisation des cartes 4
Préparation de l'atelier en distanciel avec utilisation d'un board 5
Analyse des actions proposées dans l'atelier (par capacité) **6**
Capacité 1 - Intégration continue 6
Action 10 - Intégration continue et correction - Le meilleur choix 6
Action 11 - Automatisation intégration partielle 7
Question bonus 7
Capacité 2 - Automatisation des tests 7
Action 13 - Acceptance semi automatisée 7
Action 14 - Automatisation des tests couvrant l'ensemble du cycle de vie - Le meilleur choix 8
Question bonus 8
Capacité 3 - Automatisation des déploiements 9
Action 15 - Méthodes différentes pour Dev et Ops 9
Action 16 - Processus unique et automatisé - Le meilleur choix 9
Question bonus 10
Capacité 4 - Maintenabilité du code 10
Action 17 - Modification avec processus d'approbation 10
Action 18 - Gestion autonome - Le meilleur choix 11
Question bonus 12
Capacité 5 - Architecture faiblement couplée 12
Action 19 - Architectures simples privilégiées 12
Action 20 - Architecture faiblement couplée - Le meilleur choix 13
Question bonus 13
Capacité 6 - Qualité de la documentation 14
Action 21 - Documentation durant tout le processus - Le meilleur choix 14
Action 22 - Le piège du faux agile 15
Question bonus 15
Capacité 7 - Responsabiliser équipes sur choix des outils 15
Action 23 - Autonomie totale 15
Action 24 - Autonomie guidée - Le meilleur choix 16
Question bonus 17
Capacité 8 - Développement à branche unique 17
Action 25 - Travail par branches avec plusieurs fonctionalités 17
Action 26 - Travail sur de petits changements - Le meilleur choix 17

licensed under CC BY-NC-SA 4.0

Question bonus 18
Capacité 9 - Rationaliser l'approbation des changements 18
Action 27 - Processus allégé d'approbation - Le meilleur choix 18
Action 28 - CAB pour changement à haut risques 19
Question bonus 19
Capacité 10 - Contrôle de version 20
Action 29 - Sur code applicatif et IaC seulement 20
Action 30 - Contrôle de version étendu - Le meilleur choix 20
Question bonus 21
Capacité 11 - infrastructure flexible 21
Action 31 - Passer au Cloud - Le meilleur choix 21
Action 32 - Migration Lift & shift 22
Action 33 - Cloud privé basé sur solution de virtualisation 22
Capacité 12 - Automatisation des déploiements 23
Action 35 - Travail par petits lots - Le meilleur choix 23
Action 36 - Livraison en Prod après plusieurs itérations 23
Question bonus 24
Capacité 13 - Surveillance et observabilité 24
Action 38 - Couvrir toutes les alertes et erreurs possibles 24
Action 39 - Aller plus loin que le monitoring - Le meilleur choix 24
Question bonus 25
Capacité 14 - Sécurité omniprésente 25
Action 38 - Sécurité intégré au quotidien - Le meilleur choix 25
Action 39 - Automatisation + Contrôle juste avant la livraison 26
Question bonus 26
Capacité 15 - Gestion des données de tests 26
Action 40 - Données standardisées et anonymisées 26
Action 41 - Données à la demande et automatisation - Le meilleur choix 27
Action 42 - Copie de production 27
Question bonus 28
Capacité 16 - Visibilité du travail dans la chaîne de valeur 28
Action 43 - Améliorations locales 28
Action 44 - Vision complète - Le meilleur choix 29
Question bonus 29
Capacité 17 - Retours du client 29
Action 46 - Collecte régulière - Le meilleur choix 29
Action 47 - Collecte après déploiement 30
Question bonus 30
Capacité 18 - Management visuel 30
Action 54 - Suivi d'activité 30
Action 55 - Tableaux de bord complets orientés système - Le meilleur choix 31
Question bonus 32
Capacité 25 - Les métriques comme objectif 32
Aucune des actions proposées n'est correcte 32

licensed under CC BY-NC-SA 4.0

Présentation de l’aide

Cette aide pour le facilitateur permet d’expliquer et clarifier les meilleurs choix parmi les différentes propositions d’actions proposées lors de l’atelier.

Précisions importantes:

- Afin de pouvoir tenir sur une carte, la taille du texte de chaque action a dû être limitée ce qui a nécessité de faire des choix (et donc sujets à débat) et rend donc certaines actions ambiguës ou sujet à discussion. Il est donc important de laisser la discussion ouverte, d’échanger et de se référer au site officiel (dora.dev) pour aller plus loin.
- Toutes les actions proposées ne doivent pas se voir comme une voie à suivre ou à être appliquées mais plutôt comme des cas fictifs pour présenter les résultats de DORA et surtout pour amener à l’échange et la réflexion. Il n’y a pas de modèle prêt à être suivi, ce n’est pas l’objet de l’atelier.
- Chaque analyse s’appuie avant tout sur les résultats DORA (site ou livre “Accelerate” de 2019) et pour discuter plus profondément de chaque capacité, la partie du site sur les capacités (https://dora.dev/capabilities/) apportera plus d’éclairage.
- L’atelier reste une introduction, une découverte, il n’a pas pour intention de définir une transformation ou de donner une liste d’actions à appliquer mais plutôt à permettre l’échange et le questionnement sur les meilleures pratiques à adopter et les challenges associés pour leur mise en place pour permettre de s’améliorer.
- Le chapitre concernant l’analyse des actions est donné à titre indicatif et n’a pas vocation à être suivi à la lettre, de manière à laisser les échanges ouverts.

Présentation et préparation de l’atelier

Présentation de l’atelier

Cet atelier a été présenté à Agile game Toulouse 2024 et Agile tour Toulouse 2025.
L’idée de base est de faire découvrir et mieux appréhender par petits groupes
de manière ludique les concepts et résultats issus du livre Accelerate et de DORA.

L’objectif de ce serious game est de trouver le maximum d’actions optimales permettant d’améliorer les 4 métriques DORA (et liées à des capacités). C’est l'occasion d'explorer les meilleures pratiques / leviers d’actions et leurs challenges côté DevOps, lean management et produit, ainsi que les métriques associées.

Atelier pour des groupes max de 5 à 6 personnes maximum, durée 1 heure (à adapter en fonction du nombre de cartes).

licensed under CC BY-NC-SA 4.0

Organisation de l'atelier

Une possibilité d'animation est de découper l'atelier en 3 phases (effectuées en groupes. Prévoir plusieurs facilitateurs si besoin. Durée de chaque phase à titre indicatif) avec une introduction et une conclusion avec moment d'échange avec tout le monde:

- Introduction à DORA / Accelerate et présentation de l'atelier - 8'
- Une 1ere phase d'échauffement et de découverte du déroulement de l'atelier - 8'
    - Chaque groupe découvre les actions associées à 2 capacités et sélectionne les plus pertinentes
    - Correction et débriefing
- Une 2eme phase pour découvrir les capacités orientées technique les plus pertinentes - 18'
    - Chaque groupe découvre les actions associées à 2 capacités et sélectionne les plus pertinentes
    - Correction et débriefing
- Une 3eme phase pour découvrir les capacités orientées Lean / produit les plus pertinentes - 18'
    - Chaque groupe découvre les actions associées à 2 capacités et sélectionne les plus pertinentes
    - Correction et débriefring
- Débriefing global, séance Q/R, Feedbacks - 8'

Préparation de l'atelier en présentiel avec utilisation des cartes

Il est nécessaire d'imprimer le fichier contenant l'ensemble des cartes à utiliser ainsi que des cartes bonus (cf fichier ***Cartes-pour-atelier-DORA-Accelerate-pied-au-plancher.pdf*** pour un jeu de cartes correspondant à un groupe).

Exemple de découpage de l'atelier en 3 phases:

- Une 1ere phase d'échauffement et de découverte du déroulement de l'atelier avec les cartes actions liées à 2 capacités:
  Capacités No 6 et 10
  Actions associées No 21, 22, 29, 30

- Une 2eme phase pour découvrir les capacités orientées technique les plus pertinentes avec les cartes suivantes:
  Capacités No 1, 2, 3, 4, 5, 11, 13 et 14
  Actions associées No 10, 11, 13, 14, 15, 16, 17, 18, 19, 20, 31, 32, 33, 36, 37, 38 et 39

- Une 3eme phase pour découvrir les capacités orientées Lean / produit les plus pertinentes avec les cartes suivantes:
  Capacités No 7, 9, 12, 16, 17, 18, 19, 25

licensed under CC BY-NC-SA 4.0

Actions associées No 23, 24, 27, 28, 34, 35, 43, 44, 46, 47, 54, 55, 56, 57, 59 et 60

Les autres cartes Capacités (8, 15, 20 - 26) et leurs Actions associées étant en bonus.

Exemple de disposition des cartes sur chaque table assignée à un groupe (une colonne par phase, les cartes capacités étant retournées):

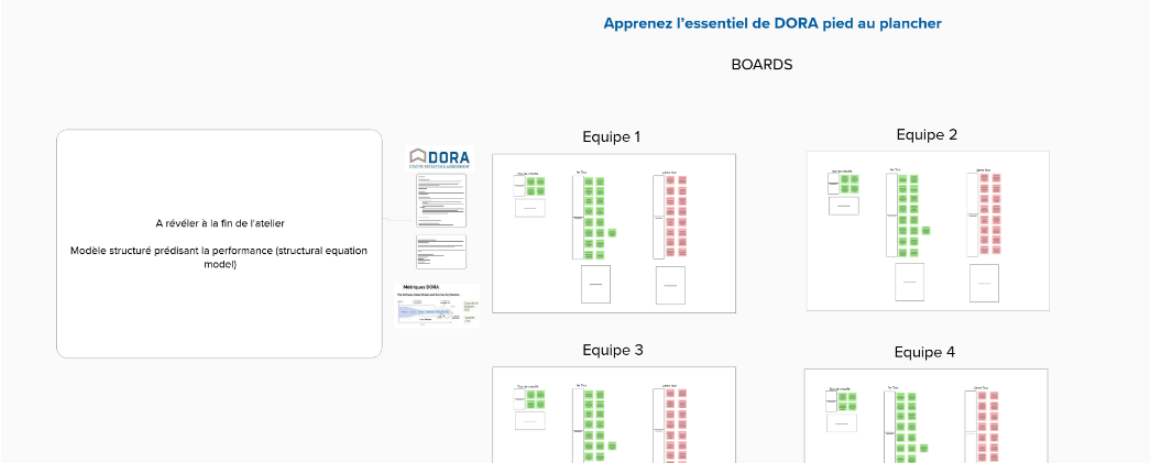
- A chaque tour, sélectionnez en équipe les meilleures actions (bandeau vert) pour améliorer les 4 métriques DORA.
- A la fin du tour, découvrez les choix optimaux en retournant les cartes “Capacités” (bandeau bleu)

Préparation de l’atelier en distanciel avec utilisation d’un board

Le board template Mural est disponible libre d’usage (licensed under CC BY-NC-SA 4.0):
https://app.mural.co/t/dev18468/template/5a027682-c5cd-45e4-9035-80b7f95781bd

licensed under CC BY-NC-SA 4.0

Exemple du template avec une colonne par phase, les cartes capacités étant retournées:



Analyse des actions proposées dans l'atelier (par capacité)

Capacité 1 - Intégration continue

Action 10 - Intégration continue et correction - Le meilleur choix

Texte

"Mise en place du build automatisé à chaque commit du code, pour la phase d'intégration, en incluant tests unitaires. En cas d'échec, les développeurs corrigent au plus vite"

.**Analyse**

Il s'agit du meilleur choix pour les raisons suivantes:

Rapidité du feedback : Les tests automatisés doivent être exécutés contre chaque commit pour donner aux développeurs un feedback immédiat sur leurs changements.

La règle de l'arrêt immédiat : Si une partie du build échoue, les développeurs doivent corriger le problème immédiatement. C'est ce qui garantit que le logiciel reste dans un état "déployable" en permanence.

Réduction de la dette technique : Travailler sur des branches à vie courte (moins d'un jour) évite les phases de "stabilisation" ou de "code freeze" douloureuses rencontrées par les performeurs moyens ou faibles.

licensed under CC BY-NC-SA 4.0

L’objectif recommandé par DORA pour les équipes est:

- D’effectuer tous les types de tests en continu tout au long du cycle de vie de livraison logicielle
- De créer et maintenir des suites de tests automatisés rapides et fiables, intégrées à vos pipelines de livraison continue.

Action 11 - Automatisation intégration partielle

Texte

“Mise en place du build automatisé pour la phase d'intégration, lancé lors de la livraison d'un ensemble de fonctionnalités.”

.**Analyse**

Le build lancé lors de la livraison d'un ensemble de fonctionnalités est un anti pattern.

Cette approche est typique des organisations qui travaillent en **grands lots (large batches)**, ce que l'étude dénonce vigoureusement.

Les risques identifiés par DORA :

- Complexité d'intégration : Attendre d'avoir un "ensemble de fonctionnalités" avant d'intégrer multiplie de façon exponentielle le temps nécessaire au débuggage et au rework.
- Variabilité et Risque : La réduction de la taille des lots est centrale au paradigme Lean ; elle réduit les risques, les coûts et la croissance des délais .
- Perte de visibilité : En effectuant des livraisons par lots, vous perdez la capacité de corréler une erreur spécifique à une modification précise du code.

Question bonus

Le fait de "corriger au plus vite" ne ralentit-il pas trop le développement des nouvelles fonctionnalités ?

L’action 11 n'est pas de l'Intégration Continue, mais de "l'Intégration Différée". La force de l’intégration continue réside dans la discipline de corriger au plus vite. Sans cette discipline, l'automatisation perd 80% de sa valeur.

Capacité 2 - Automatisation des tests

Action 13 - Acceptance semi automatisée

Texte

licensed under CC BY-NC-SA 4.0

“Automatisation des tests unitaires et d'intégration en laissant les tests d'acceptance semi automatisés.”

.**Analyse**

Cette option propose de laisser les tests d'acceptation en "semi-automatisé". Dans le cadre d'une transformation Lean/DevOps, c'est souvent un goulot d'étranglement majeur.

Le "Fuzzy Front End" : Si la recette utilisateur reste manuelle ou semi-manuelle, vous créez une rupture dans la boucle de feedback. Les recherches montrent que la durée de livraison (lead time) est fortement corrélée à l'automatisation des tests. Un processus semi-automatisé réintroduit de la variabilité et de l'attente.

Implication des développeurs : Une découverte clé de DORA est que lorsque les développeurs créent et maintiennent eux-mêmes les tests d'acceptation, le code devient plus testable (via le TDD) et les équipes se sentent plus responsables de la qualité globale.

Action 14 - Automatisation des tests couvrant l'ensemble du cycle de vie - Le meilleur choix

Texte

“Mise en place de l'automatisation complète de l'ensemble des tests (unitaires, d'intégration, d'acceptance / recette utilisateurs).”

.**Analyse**

Le modèle DORA est catégorique : pour atteindre les bénéfices de la livraison continue, l'automatisation doit couvrir l'ensemble du cycle de vie des tests.

Il est faux de croire que cette option signifie qu’il ne doit pas y avoir de testeurs humains:

1. **Automatisez le répétitif :** Tout ce qui peut être vérifié par une machine (critères d'acceptation, non-régression) doit l'être.
2. **Libérez l'humain pour la valeur :** Les testeurs doivent effectuer des tests manuels dits **exploratoires**, d'utilisabilité et de sécurité créative. Ces tests ne sont pas "semi-automatisables", ils demandent de l'intuition.

Question bonus

L'automatisation des tests d'acceptation entraîne-t-elle une charge de travail insurmontable par rapport aux tests unitaires ?

L'automatisation complète des tests (unitaires, intégration, acceptation) est un prédicteur statistique de :

- Une meilleure performance de livraison de logiciel (vitesse et stabilité).

licensed under CC BY-NC-SA 4.0

- Une réduction du taux d'échec des changements (change fail rate).
- Une diminution de la souffrance liée aux déploiements et du burnout des équipes.

Si vous n’avez pas de suite de tests automatisés: commencez par écrire quelques tests unitaires et d’acceptation couvrant les fonctionnalités à forte valeur ajoutée de votre système. Assurez-vous que ces tests soient fiables. De cette façon, s'ils échouent, vous saurez qu'il y a un réel problème, et s'ils réussissent, vous aurez l'assurance que le système ne présente aucune anomalie grave. Ensuite, veillez à ce que chaque nouvelle fonctionnalité soit couverte par des tests. Ces tests doivent s'exécuter rapidement afin de fournir un feedback aux développeurs le plus tôt possible. Vos tests devraient s'exécuter avec succès au moins une fois par jour. À terme, si vous disposez de tests de performance et d'acceptation, les développeurs devraient en recevoir les résultats quotidiennement.

Capacité 3 - Automatisation des déploiements

Action 15 - Méthodes différentes pour Dev et Ops

Texte

“Déploiement automatisé en tenant compte des configurations différentes sur chaque environnement, en laissant aux développeurs et aux opérations leurs propres méthodes de déploiement.”

.**Analyse**

Des méthodes différentes pour le Dev et l'Ops est un anti pattern côté DevOps, ce qui entraîne:

Silos et frictions : Laisser les développeurs et les opérations utiliser leurs propres méthodes de déploiement recrée une barrière fonctionnelle. Cela conduit souvent les développeurs à jeter du code de mauvaise qualité par-dessus le mur aux opérations.

Risque d'échec accru : Des méthodes de déploiement divergentes signifient que le premier déploiement "réel" avec la méthode des opérations se produit souvent en production. C'est la recette idéale pour des erreurs de configuration et des pannes.

Pénibilité du déploiement (Deployment Pain) : La recherche montre que des processus de déploiement complexes et fragiles augmentent considérablement la souffrance des équipes et le risque de burnout.

Action 16 - Processus unique et automatisé - Le meilleur choix

Texte

licensed under CC BY-NC-SA 4.0

“Automatisation du déploiement en utilisant le même process de déploiement pour chaque environnement (production inclus).”

.**Analyse**

Un processus unique et automatisé pour tous les environnements.

La recherche DORA montre que la capacité de déploiement automatisé est un prédicteur clé de la performance. L'utilisation du même processus pour chaque environnement est fondamentale pour plusieurs raisons :

- **Réduction de la variabilité :** L'un des objectifs du Lean est de réduire la variabilité. Utiliser le même mécanisme en test qu'en production permet de valider le processus de déploiement lui-même des centaines de fois avant qu'il ne touche la production.
- **Fiabilité par la répétition :** Pour les hauts performeurs, le déploiement doit être une activité de routine, simple et prévisible. Cela n'est possible que si le chemin vers la production est standardisé.
- **Source unique de vérité :** Tout le processus, y compris la configuration, doit être géré via le contrôle de version et appliqué de manière automatisée.
- **Déploiement en un clic :** Dans les organisations matures, le déploiement vers n'importe quel environnement, y compris la production, peut souvent être déclenché par une seule commande .

Question bonus

Quels sont les avantages pour les équipes "Ops" d'utiliser les mêmes outils que les développeurs pour la production ?

Le déploiement est un résultat systémique qui nécessite une collaboration étroite entre tous les acteurs. Avoir un processus unique force cette collaboration dès le début du cycle de développement.

Capacité 4 - Maintenabilité du code

Action 17 - Modification avec processus d’approbation

Texte

“Chaque équipe a la possibilité de rechercher et modifier si besoin le code maintenu par d'autres équipes avec un process d'approbation (dépendances), Ceci afin d'améliorer la maintenabilité du code.”

.**Analyse**

La modification du code tiers avec processus d'approbation.

licensed under CC BY-NC-SA 4.0

Bien que l'idée de pouvoir modifier le code d'une autre équipe (InnerSource) soit noble, l'ajout d'un **"processus d'approbation"** et la mention de **"dépendances"** sont des signaux d'alerte majeurs dans le cadre DORA.

Les problèmes identifiés par l'étude :

- **Obstacles à la vitesse :** Devoir dépendre d'autres équipes pour faire des changements ou obtenir des approbations est une barrière significative à l'augmentation de la fréquence de livraison.
- **Échec des contrôles externes :** Les recherches montrent que les processus d'approbation par un organisme externe (comme un CAB ou une autre équipe) ne renforcent pas la stabilité, mais ralentissent considérablement le flux.
- **Saturation de la bande passante :** Si les équipes doivent constamment discuter des décisions d'implémentation à cause d'un code partagé et dépendant, la bande passante de communication est saturée au détriment des objectifs stratégiques

Action 18 - Gestion autonome - Le meilleur choix

Texte

“Chaque équipe gère son code de manière exclusive afin de permettre une meilleure gestion du refactoring, de sa maintenabilité et de sa stabilité.”

Analyse

La gestion autonome (indépendante) de son propre périmètre.

Dans le modèle DORA, la maintenabilité n'est pas une fin en soi, mais un moyen d'atteindre la performance de livraison. Le livre souligne que la caractéristique architecturale la plus déterminante est le **couplage faible (loose coupling)**.

Pourquoi c'est la meilleure option selon la science :

- **Indépendance des changements :** Les équipes les plus performantes peuvent effectuer des changements d'envergure sur la conception de leur système sans avoir besoin de la permission de quelqu'un à l'extérieur de l'équipe.
- **Réduction des dépendances :** Elles peuvent terminer leur travail sans avoir besoin de communiquer ou de se coordonner avec des personnes extérieures à l'équipe.
- **Testabilité et Déploiement :** Une architecture découplée permet de tester et de déployer un service indépendamment des autres services dont il dépend.
- **Scalabilité :** Cette approche permet d'augmenter la productivité linéairement, voire mieux, à mesure que l'organisation grandit.

Cette proposition ne prône pas l’isolationisme, mais **l'autonomie de décision**:

L'interface est le contrat : Les équipes doivent interagir via des APIs et des "bounded contexts" bien définis. Tant que le contrat est respecté, chaque équipe doit être libre de refactorer son code comme elle l'entend pour en assurer la maintenabilité.

licensed under CC BY-NC-SA 4.0

Équipes cross-fonctionnelles : Pour que cette gestion exclusive fonctionne, l'équipe doit posséder toutes les compétences nécessaires (dev, test, ops, sécurité) pour ne pas créer de dépendances externes.

Standardisation invisible : L'organisation peut proposer des bibliothèques et des outils pré-approuvés (comme le fait Google ou Amazon) pour faciliter le travail, mais sans les imposer si cela nuit à la performance de l'équipe.

Question bonus

Avec cette gestion autonome / “exclusive", ne risque-t-on pas de créer des doublons de code ou des incohérences technologiques entre les équipes ?

La "maintenabilité" dans Accelerate est intimement liée à la testabilité sans environnement intégré coûteux. Si une équipe gère son code de manière "exclusive", elle peut automatiser ses propres tests d'acceptation et garantir sa qualité sans attendre personne.

Capacité 5 - Architecture faiblement couplée

Action 19 - Architectures simples privilégiées

Texte

“Favoriser les architectures simples et éprouvées basées sur frontend, backend et base de donnée permettant de livrer facilement en diminuant l'effort de montée en compétence des équipes.”

Analyse

Privilégier les architectures "simples et éprouvées" (N-tier).

Bien que la simplicité soit une vertu, cette proposition passe à côté de la découverte majeure du livre.

- **Le type de système importe peu :** L'étude a révélé qu'il n'y a pas de corrélation significative entre le type de système (qu'il s'agisse de systèmes "greenfield", de "systems of record" ou même de mainframes) et la performance de livraison .
- **L'erreur du focus technologique :** Ce ne sont pas les technologies ou les schémas classiques (frontend/backend/DB) qui prédisent la performance, mais les **caractéristiques** architecturales : la testabilité et la déployabilité indépendantes.
- **L'effort de montée en compétence :** *Accelerate* souligne que l'investissement dans le développement des capacités des équipes (comme apprendre à découper des domaines complexes) est essentiel pour réussir la transformation DevOps .

licensed under CC BY-NC-SA 4.0

Action 20 - Architecture faiblement couplée - Le meilleur choix

Texte

“Mettre en place une architecture faiblement couplée pour permettre aux équipes de travailler de manière indépendante, sans dépendre d'autres équipes pour le support et les services.”

Analyse

L'architecture faiblement couplée pour l'indépendance des équipes.

La recherche a démontré que l'architecture est l'un des prédicteurs les plus puissants de la performance de livraison de logiciels. L'objectif d'une architecture faiblement couplée n'est pas seulement technique, il est avant tout **organisationnel**.

Pourquoi c'est la meilleure option :

- **Autonomie radicale :** Elle permet aux équipes d'effectuer des changements d'envergure sur la conception de leur système sans avoir besoin de la permission de personnes extérieures à l'équipe.
- **Réduction des dépendances :** Les équipes peuvent terminer leur travail (du design au déploiement) sans communication ou coordination intensive avec d'autres équipes.
- **Livraison à la demande :** Une équipe peut déployer et livrer son produit ou service indépendamment des autres services dont il dépend.
- **Scalabilité de la productivité :** C'est cette capacité qui permet aux "High Performers" d'augmenter leur nombre de déploiements par développeur et par jour de manière linéaire, voire supérieure, à mesure que la taille de l'organisation augmente .

L'organisation doit faire évoluer sa structure d'équipe et son architecture système en parallèle. Pour que cette proposition fonctionne :

1. **Architecture :** Utilisez des contextes délimités (bounded contexts) et des APIs pour découper les grands domaines en unités plus petites.
2. **Équipes :** Assurez-vous que les équipes de livraison sont **cross-fonctionnelles**, possédant toutes les compétences nécessaires (conception, dev, test, ops, sécurité) sur le même banc.

Question bonus

Quel est l’impact organisationnel d’avoir une architecture faiblement couplée ?

Pour que cela fonctionne, l'organisation doit faire évoluer sa structure d'équipe et son architecture système en parallèle. Il faut entre autre:

1. **Architecture :** Utilisez des contextes délimités (bounded contexts) et des APIs pour découper les grands domaines en unités plus petites.

licensed under CC BY-NC-SA 4.0

2. **Équipes :** Assurez-vous que les équipes de livraison sont **cross-fonctionnelles**, possédant toutes les compétences nécessaires (conception, dev, test, ops, sécurité) sur le même banc.

Capacité 6 - Qualité de la documentation

Action 21 - Documentation durant tout le processus - Le meilleur choix

Texte

“La qualité de la documentation est assurée en l'intégrant dans le processus de développement, avec la mise en place explicite de consignes pour sa mise à jour, les propriétaires de ces ressources étant identifiés.”

Analyse

L'intégration de la documentation dans le processus de développement.

Cette approche suit les principes fondamentaux de la **Continuous Delivery (CD)** et du **Lean Management** :

- **Construire la qualité dès le départ (Build Quality In) :** DORA insiste sur le fait qu'il ne faut pas dépendre de l'inspection finale, mais intégrer la qualité directement dans le processus de production. Traiter la documentation comme une partie intégrante du "Done" (terminé) au même titre que le code suit cette logique.
- **Réduction du travail non planifié (Unplanned Work) :** L'ouvrage note que répondre à des demandes urgentes de documentation (souvent pour des audits ou du support) est considéré comme du travail non planifié ou du "rework" s'il n'est pas géré correctement.
- **Responsabilité partagée :** En identifiant les propriétaires et en rendant les consignes explicites, l'organisation s'assure que "tout le monde est responsable" de la qualité globale du système, un trait caractéristique des cultures génératives.
- **Soutien à l'apprentissage (Climate for learning) :** Une documentation de qualité et à jour est un levier essentiel pour une culture qui encourage l'apprentissage et le partage d'informations.

La documentation est une **capacité de transmission**. Pour qu'elle soit efficace selon DORA, elle doit être :

1. **Découvrable :** Facile à trouver pour ceux qui en ont besoin.
2. **Actionnable :** Permettant de réaliser une tâche sans aide extérieure.
3. **À jour :** Maintenue avec la même rigueur que le code source (concept de *Documentation as Code*).

licensed under CC BY-NC-SA 4.0

Action 22 - Le piège du faux agile

Texte

“Un logiciel fonctionnel étant plus important qu’une documentation exhaustive, les équipe se concentrent sur un niveau suffisant de détails dans les user stories et/ou sur la mise à jour de la documentation via outil du style Confluence.”

.**Analyse**

Le logiciel fonctionnel comme excuse à une documentation insuffisante.

Cette proposition reflète ce que l'ouvrage appelle parfois le **"Faux Agile"**.

- **Mauvaise interprétation du Manifeste Agile :** Bien que le manifeste privilégie le logiciel fonctionnel, l'étude DORA montre que les "High Performers" ne font pas de compromis entre vitesse et qualité. Une documentation négligée crée une dette technique qui ralentira les équipes plus tard.
- **Goulot d'étranglement de l'information :** Se contenter des user stories ou de mises à jour sporadiques sur un wiki (style Confluence) sans processus rigoureux mène souvent à une information obsolète. Cela nuit à la "visibilité du flux de travail", une capacité pourtant essentielle à la performance.
- **Augmentation de la pénibilité :** L'absence de documentation claire sur les systèmes augmente la "pénibilité du déploiement" (Deployment Pain) et le stress des équipes de maintenance, ce qui est un facteur direct de burnout.

Question bonus

L'intégration de la documentation dans le cycle de développement ne va-t-elle pas ralentir la "vitesse" perçue des développeurs ?

La documentation est une **capacité de transmission**. Pour qu'elle soit efficace selon DORA, elle doit être :

1. **Découvrable :** Facile à trouver pour ceux qui en ont besoin.
2. **Actionnable :** Permettant de réaliser une tâche sans aide extérieure.
3. **À jour :** Maintenue avec la même rigueur que le code source (concept de *Documentation as Code*).

Capacité 7 - Responsabiliser équipes sur choix des outils

Action 23 - Autonomie totale

Texte

licensed under CC BY-NC-SA 4.0

“Donner à chaque équipe la liberté de choisir ses outils afin de permettre une autonomie complète sans être freiné par des standards imposés.”

.**Analyse**

L'autonomie totale sans aucun standard.

Option qui semble séduisante pour les développeurs, mais qui présente des risques :

- **Fragmentation excessive :** Sans aucune base commune, l'organisation risque de se retrouver avec autant de technologies que d'équipes, rendant la mobilité des collaborateurs et la maintenance à long terme impossibles.
- **Focalisation sur l'accessoire :** Les architectes ne devraient pas se concentrer sur l'outil lui-même, mais sur les résultats qu'il permet d'atteindre. L'autonomie complète peut parfois amener les équipes à passer trop de temps à comparer des outils au lieu de livrer de la valeur.
- **Désengagement organisationnel :** La recherche montre que la satisfaction au travail dépend du fait d'avoir les bons outils pour faire son travail. Si chaque équipe doit tout réinventer de zéro, la charge cognitive augmente, ce qui peut mener au burnout.

Action 24 - Autonomie guidée - Le meilleur choix

Texte

“Laisser le choix des outils aux équipes avec une base commune définie par les équipes multi domaines pouvant évoluer et si besoin un process pour gérer les exceptions.”

.**Analyse**

L'autonomie guidée (le concept de "Paved Road").

Le choix des outils par les équipes est un prédicteur de la performance de livraison de logiciels et de la performance organisationnelle. Cette proposition l'emporte car elle intègre des nuances critiques :

- **Standardisation à valeur ajoutée :** DORA reconnaît qu'il existe une place pour la standardisation, notamment autour de l'architecture et de la configuration de l'infrastructure. Une base commune permet de réduire la complexité environnementale et d'assurer le partage des compétences.
- **Adoption volontaire :** DORA souligne que lorsque les outils fournis (la base commune) facilitent réellement la vie des ingénieurs, ceux-ci les adoptent de leur plein gré. C'est une approche bien plus efficace que d'imposer des outils choisis pour la convenance de parties prenantes externes.
- **Sécurité et conformité facilitées :** L'existence d'une base commune permet aux équipes de sécurité de fournir des bibliothèques et des chaînes d'outils

licensed under CC BY-NC-SA 4.0

pré-approuvées et faciles à consommer. Cela aide les équipes à "shifter à gauche" (intégrer la sécurité tôt) sans ralentir leur flux.

Bien que le livre Accelerate insiste lourdement sur l'autonomie, il prône un équilibre subtil entre la liberté de choix et la cohérence globale de l'entreprise.

Question bonus

Quelle est l'importance du choix de l'outil ?

Remarques:

Peu importe l'outil choisi, ce qui compte pour DORA est de savoir si l'équipe peut déployer de manière indépendante, sans dépendances externes.

Il est crucial d'avoir un processus pour gérer les exceptions: Si la "base commune" devient un frein, l'équipe doit avoir la liberté d'expérimenter de nouvelles approches pour résoudre ses problèmes spécifiques.

Capacité 8 - Développement à branche unique

Action 25 - Travail par branches avec plusieurs fonctionalités

Texte

"Les développeurs travaillent via des branches et fusionnent ensuite leur code pour livraison de plusieurs fonctionnalités."

.**Analyse**

Le travail par branches isolées pour plusieurs fonctionnalités.

Cette option correspond souvent à ce que l'on appelle le "GitHub Flow" ou le "GitFlow" mal maîtrisé. Bien que populaire, la recherche DORA montre que cette méthode ralentit la performance pour les raisons suivantes :

- **Enfer de l'intégration :** Attendre la fin de "plusieurs fonctionnalités" avant de fusionner crée des conflits de fusion complexes et décourage le refactoring régulier.
- **Manque de communication :** Le maintien de plusieurs branches à longue durée de vie nuit à la communication au sein de l'équipe.
- **Instabilité :** Plus la durée de vie d'une branche est longue, plus le risque de découvrir des erreurs tardivement est élevé, ce qui augmente le travail non planifié et le taux d'échec des changements.

Action 26 - Travail sur de petits changements - Le meilleur choix

Texte

licensed under CC BY-NC-SA 4.0

“Les développeurs travaillent sur de petites fonctionnalités et fusionnent ensuite leur code sur la branche master au moins une fois par jour.”

.**Analyse**

Travailler sur de petits changements fusionnés quotidiennement.

Cette approche est un prédicteur direct de la haute performance en livraison de logiciels. Selon les données de DORA */ Accelerate*, le développement à branche unique se caractérise par :

- **La brièveté des branches :** Les branches et les forks ont des durées de vie très courtes, typiquement **moins d'une journée**, avant d'être intégrés dans la branche principale (*master* ou *trunk*).
- **La petite taille des lots :** Les développeurs décomposent le travail en petites fonctionnalités qui peuvent être terminées en quelques heures ou une journée.
- **La fluidité :** Les équipes performantes n'ont jamais de périodes de "gel du code" (*code freeze*) ou de phases de stabilisation manuelles.

Question bonus

Ne va-t-on pas "casser la branche master" en fusionnant le code tous les jours ?

Adopter la branche unique n'est pas seulement une préférence technique, c'est un levier de transformation culturelle et business :

- **Réduction du Burnout :** Le développement basé sur le tronc est corrélé à une réduction de la souffrance lors des déploiements et du burnout des équipes.
- **Vitesse et Qualité :** Les hauts performeurs ont les temps d'intégration les plus courts, ce qui leur permet de livrer plus vite tout en maintenant une meilleure stabilité .
- **Culture Générative :** En intégrant le code quotidiennement, l'équipe partage la responsabilité de la qualité globale, ce qui renforce la confiance et la collaboration .

Capacité 9 - Rationaliser l'approbation des changements

Action 27 - Processus allégé d’approbation - Le meilleur choix

Texte

“Mise en place d'un process allégé d'approbation des changements basé sur revue de pair, pair programming combiné à un pipeline de déploiement détectant et rejetant les mauvais changements.”

.**Analyse**

licensed under CC BY-NC-SA 4.0

Un processus allégé basé sur la revue par les pairs et l'automatisation.

La recherche DORA a identifié qu'un processus d'approbation des changements "léger" est un prédicteur de la haute performance. La proposition 1 est la meilleure car :

- **Performance supérieure** : Les équipes utilisant la revue par les pairs (comme le pair programming ou la revue de code intra-équipe) atteignent une performance de livraison de logiciels plus élevée que celles dépendant d'un comité externe.
- **Efficacité systémique** : L'utilisation d'un pipeline de déploiement automatisé permet de détecter et rejeter les mauvais changements de manière beaucoup plus fiable et rapide qu'une inspection humaine.
- **Culture de confiance** : Ce modèle renforce la responsabilité de l'équipe et favorise une culture générative où la qualité est intégrée au travail quotidien.

Action 28 - CAB pour changement à haut risques

Texte

“Modification du process d'approbation des changements en ne considérant que les changes à haut risque lors du CAB (Change Advisory Board).”

.**Analyse**

Ne garder le CAB (Change Advisory Board) que pour les changements à haut risque.

Bien que cela semble être une amélioration par rapport à un CAB systématique, la science d'*Accelerate* montre que cette approche n'est pas efficace :

- **Absence de corrélation** : L'étude a révélé que l'approbation limitée aux changements à haut risque n'est **pas corrélée** avec la performance de livraison de logiciels.
- **Ralentissement sans gain de stabilité** : Les approbations externes (comme un manager ou un CAB) sont négativement corrélées avec la vitesse (délai de livraison, fréquence de déploiement) et n'ont **aucune corrélation** avec la baisse du taux d'échec. En clair : cela ralentit tout sans rendre le système plus stable.
- **Le "Théâtre de la gestion des risques"** : L'ouvrage qualifie ces revues externes de "théâtre", car des personnes non familières avec le code ne peuvent pas réellement évaluer l'impact d'un changement complexe sur un système de production.

Question bonus

Supprimer le CAB n'augmente-t-il pas le risque de pannes en production en enlevant la "séparation des tâches" (segregation of duties) et pose des problèmes pour conformité ?

La réponse de DORA :

licensed under CC BY-NC-SA 4.0

- L'auditabilité automatisée : Un pipeline de déploiement offre un historique complet et infalsifiable (qui a approuvé quoi, quand, et quels tests ont été passés), ce qui est bien plus précieux pour un auditeur qu'un compte-rendu de réunion CAB.
- La revue par les pairs suffit : Dans les industries réglementées, la séparation des tâches peut être satisfaite par le fait qu'une personne différente de l'auteur du changement valide le code (via une Pull Request par exemple) avant ou immédiatement après le commit.

Capacité 10 - Contrôle de version

Action 29 - Sur code applicatif et IaC seulement

Texte

“Mise en place et utilisation d'un outil de contrôle de version du code de l'application ainsi que le code permettant de provisionner l'infrastructure (IaC).”

.**Analyse**

Se limiter au code applicatif et à l'IaC.

Bien que cette proposition soit un bon début, elle crée des zones d'ombre (blind spots) qui ralentissent la performance :

- **Fragilité des configurations :** Si les configurations (fichiers `.env`, variables de déploiement) ne sont pas versionnées, l'équipe subit un "glissement de configuration" (*configuration drift*), ce qui rend les déploiements imprévisibles.
- **Goulots d'étranglement DB :** Sans les scripts de schémas DB dans le contrôle de version, les changements de base de données restent souvent manuels et deviennent des obstacles majeurs à la livraison continue.
- **Manque de reproductibilité :** Le modèle DORA exige qu'il soit possible de provisionner les environnements, de construire, de tester et de déployer de façon entièrement automatisée **uniquement** à partir des informations stockées dans le contrôle de version .

Action 30 - Contrôle de version étendu - Le meilleur choix

Texte

“Mise en place et utilisation d'un outil de contrôle de version du code applicatif, de la configuration, des scripts de création des schémas DB, du code servant à créer les environnements et des packages …”

.**Analyse**

licensed under CC BY-NC-SA 4.0

Le contrôle de version étendu à l'ensemble des artefacts de production.

L'étude définit le contrôle de version comme l'utilisation d'un système (tel que GitHub ou Subversion) pour **tous les artefacts de production** . L'option 2 est supérieure car elle inclut des éléments critiques souvent oubliés :

- **La configuration est la clé :** L'une des découvertes les plus surprenantes de la recherche est que le maintien de la configuration du système et de l'application dans le contrôle de version est **plus fortement corrélé** à la performance de livraison de logiciels que le simple maintien du code applicatif.
- **Scripts et Schémas :** Inclure les scripts de création d'environnements et les schémas de base de données permet de garantir que l'état du système peut être reproduit de manière automatisée à partir du contrôle de version.
- **Packages et Dépendances :** La gestion des packages assure la reproductibilité des builds et évite les pannes liées à des composants externes qui changent ou disparaissent.

Question bonus

Quelles sont les difficultés à inclure les scripts de base de données (schémas DB) dans les dépôts de code, quelles sont les actions possibles pour y arriver ?

Capacité 11 - infrastructure flexible

Action 31 - Passer au Cloud - Le meilleur choix

Texte

“Passer au Cloud (AWS, GCP, Azure, OVH, public / privé ou hybride) pour bénéficier d'un service d'infra à la demande, de la scalabilité pour des applications devenues Cloud natives.”

.**Analyse**

Cette option l'emporte car elle met l'accent sur les **capacités** (l'infrastructure à la demande, la scalabilité) et la transformation technique (**Cloud Native**).

Le site Dora.dev démontre que ce n'est pas "l'endroit" où vous hébergez vos données qui importe le plus, mais la manière dont vous y accédez. Les chercheurs utilisent la définition du **NIST** (National Institute of Standards and Technology) pour définir un "vrai" Cloud performant selon 5 critères :

- **Libre-service à la demande** (On-demand self-service).
- **Accès large au réseau** (Broad network access).
- **Mise en commun des ressources** (Resource pooling).
- **Élasticité rapide** (Rapid elasticity).

licensed under CC BY-NC-SA 4.0

- **Service mesuré** (Measured service).

Ce sont ces caractéristiques qui permettent aux équipes de déployer de manière indépendante.

Action 32 - Migration Lift & shift

Texte

“Migrer l'infrastructure sur cloud Public (AWS, GCP, Azure) en Lift and shift sans avoir à refondre les applications et évitant ainsi une longue montée en compétence des équipes”

.**Analyse**

Le piège du "Lift and Shift" : Il s’agit d’un anti-pattern classique dans l'étude DORA.

Absence de bénéfices : La recherche montre que déplacer des applications vers le Cloud sans changer leur architecture ou les pratiques techniques (le fameux "Lift and Shift" sans refonte) n'apporte que très peu de gains de performance de livraison de logiciel.

Compétences : Éviter la montée en compétence est une erreur stratégique. L'investissement dans les personnes et l'apprentissage est un prédicteur direct de la réussite d'une transformation.

Action 33 - Cloud privé basé sur solution de virtualisation

Texte

“Finaliser la mise en place d'un cloud privé basé sur une solution de virtualisation pour fournir infrastructure à la demande, scalabilité et raccourcir de manière importante la livraison de l'infrastructure.”

.**Analyse**

Bien que la recherche indique que la haute performance soit possible sur tout type de système (y compris mainframe ou privé), cette option est souvent limitée :

- **Dépendances :** Dans beaucoup d'organisations, le cloud privé reste "bureaucratique". Si le développeur doit encore ouvrir un ticket pour obtenir une VM, ce n'est pas du Cloud selon DORA.
- **Complexité de maintenance :** Cela demande à l'organisation de devenir un fournisseur d'infrastructure, ce qui détourne souvent l'énergie du cœur de métier.

Question bonus

licensed under CC BY-NC-SA 4.0

Capacité 12 - Automatisation des déploiements

Action 35 - Travail par petits lots - Le meilleur choix

Texte

“Travailler par petit lots de fonctionnalités pour les livrer sur les différents environnements jusqu'en production puis passer aux lots suivants.”

.**Analyse**

Travailler par petits lots livrés jusqu'en production.

Cette approche est un pilier fondamental de la performance technologique pour plusieurs raisons scientifiques identifiées par DORA :

- **Réduction de la taille des lots :** C'est un élément central du paradigme Lean. Réduire la taille des lots réduit les temps de cycle, la variabilité du flux et les risques.
- **Feedback rapide :** Travailler en petits lots permet d'obtenir un feedback essentiel sur le travail effectué afin de pouvoir corriger le tir rapidement.
- **Livraison Continue (CD) :** La CD est une pratique où le logiciel est dans un état déployable tout au long de son cycle de vie, permettant de déployer en production à tout moment, sur demande.
- **Économie de la livraison :** Un objectif clé est de modifier l'économie du processus pour que le coût de déploiement de changements individuels soit très bas.

Action 36 - Livraison en Prod après plusieurs itérations

Texte

“Livrer de manière itérative un petit nombre de fonctionnalités en environnement d'intégration puis de qualification puis livrer après quelques itérations l'ensemble en production.”

.**Analyse**

Livrer l'ensemble en production après quelques itérations.

Bien que la proposition 2 semble "Agile" car elle mentionne des itérations, elle réintroduit le concept de **"grands lots"** au moment le plus critique : la mise en production.

- **Le piège du regroupement :** Attendre "quelques itérations" pour livrer un ensemble en production augmente artificiellement la taille du lot de déploiement.
- **Augmentation du risque :** Plus le lot de changements est important, plus le risque d'échec est élevé et plus il est difficile d'identifier la cause racine d'un incident en production.

licensed under CC BY-NC-SA 4.0

- **Délai de livraison (Lead Time) allongé :** En stockant des fonctionnalités terminées en environnement de qualification au lieu de les mettre en production, vous augmentez votre délai de livraison, ce qui pénalise votre score de performance DORA .
- **Instabilité :** Les recherches montrent que les "Low Performers" ont tendance à accumuler les changements, ce qui mène à des déploiements plus douloureux et des temps de restauration plus longs.

Question bonus

Impacts sur métriques vitesse de livraison et stabilité ?

Y a-t-il des craintes concernant la validation métier (recette) s'ils doivent livrer chaque petit lot directement jusqu'en production ?

Capacité 13 - Surveillance et observabilité

Action 38 - Couvrir toutes les alertes et erreurs possibles

Texte

“Amélioration de l'outillage de monitoring basé sur métriques et logs pour couvrir toutes les alertes et erreurs possibles.”

.**Analyse**

Vouloir couvrir "toutes les alertes et erreurs possibles".

C'est une erreur que beaucoup d'équipes commettent en pensant bien faire. Dans le cadre DORA, cette approche présente plusieurs défauts :

- **Le bruit vs le signal :** Tenter de couvrir "toutes" les alertes mène inévitablement à la "fatigue d'alerte" (*alert fatigue*). DORA souligne que l'information doit être présentée de manière à être utilisée efficacement par le destinataire.
- **Qualité de l'information :** Trop d'alertes inutiles violent le principe de "bonne information" défini par Westrum (elle doit répondre aux questions que le receveur se pose et être opportune).
- **Focalisation sur l'outil, pas le résultat :** Améliorer l'outillage sans changer l'usage (passer de la réaction à la décision) ne suffit pas à améliorer la performance de livraison de logiciels.

Action 39 - Aller plus loin que le monitoring - Le meilleur choix

Texte

licensed under CC BY-NC-SA 4.0

“Mise en place d'outils d'observabilité en supplément du monitoring pour aider à analyser la dégradations des services ainsi que la planification des capacités.”

.**Analyse**

Utiliser les données pour la décision, la dégradation et la planification.

La recherche DORA définit le monitoring non pas comme un simple système d'alerte, mais comme un levier stratégique. Cette option est supérieure car :

- **Au-delà de l'alerte :** DORA précise que le monitoring doit aller "au-delà de biper les gens quand les choses tournent mal". Il doit servir à prendre des décisions commerciales quotidiennes.
- **Approche proactive :** L'analyse de la dégradation des services permet de détecter et de mitiger les problèmes de manière préventive avant qu'ils ne deviennent des pannes totales.
- **Planification des capacités :** L'utilisation des données pour la planification montre une maturité où l'infrastructure n'est plus réactive mais alignée sur les besoins futurs de l'organisation.
- **Impact sur la culture :** Un monitoring efficace, combiné à des indicateurs visuels, aide à créer une culture générative où l'information est activement recherchée pour améliorer le système .

Question bonus

Le monitoring sert-il uniquement à détecter les problèmes ? Quel est l’intérêt d’avoir un feedback rapide sur la qualité et la déploybilité de son système ?

Concernant les logs: Faut-il "avoir beaucoup de logs" ou "avoir une bonne visibilité sur la santé du service" ?

Capacité 14 - Sécurité omniprésente

Action 38 - Sécurité intégré au quotidien - Le meilleur choix

Texte

“Sécurité intégrée au quotidien des équipes, de la conception à la livraison incluant les équipes sécurité et l'intégration d'outils.”

.**Analyse**

La sécurité comme partie intégrante du travail quotidien.

La recherche DORA montre que la sécurité n'est pas une phase, mais une **capacité continue**. L'option 1 est la meilleure car elle remplit les critères clés des "High Performers" :

licensed under CC BY-NC-SA 4.0

- **Intégration précoce (Shift Left) :** La sécurité est intégrée dès la phase de conception (design) et non à la fin.
- **Responsabilité partagée :** Les experts en sécurité ne sont plus des contrôleurs isolés, mais collaborent avec les équipes de développement tout au long du cycle (participation aux démonstrations, aide à l'automatisation des tests).
- **Autonomie des équipes :** L'organisation fournit aux développeurs des bibliothèques et des chaînes d'outils (toolchains) pré-approuvées, leur permettant de "bien faire" sans ralentir leur flux.
- **Impact mesurable :** En agissant ainsi, les équipes de haute performance passent **50 % moins de temps** à corriger des problèmes de sécurité que les équipes de faible performance.

Action 39 - Automatisation + Contrôle juste avant la livraison

Texte

“Accélérer les validations de sécurité juste avant livraison et automatiser les contrôles de code selon standards securité.”

.**Analyse**

Le piège du contrôle "juste avant livraison".

Même si elle propose de l'automatisation, la proposition 2 reste ancrée dans un modèle traditionnel de "goulot d'étranglement" :

- **Le coût du rework :** Impliquer la sécurité "juste avant livraison" est souvent douloureux et coûteux. Si un défaut architectural majeur est découvert à ce stade, l'équipe doit faire un retour arrière massif (rework).
- **Le bottleneck (Goulot d'étranglement) :** Dans un monde où l'on déploie plusieurs fois par jour, les équipes de sécurité n'ont physiquement pas la capacité de valider chaque changement manuellement ou via un processus séparé en fin de chaîne .
- **Culture de méfiance :** Cette approche renforce les silos fonctionnels, là où DevOps cherche à créer une culture générative de confiance et de partage des responsabilités.

Question bonus

Les équipes de développement sont-elles aujourd'hui capables d'assumer cette responsabilité ? Faut-il considérer la sécurité comme le problème des équipes sécurité ?

Capacité 15 - Gestion des données de tests

Action 40 - Données standardisées et anonymisées

licensed under CC BY-NC-SA 4.0

Texte

“Utiliser un jeu standard de données de tests anonymisées permettant d'obtenir des résultats de référence reproductibles.”

.**Analyse**

Le jeu standardisé (Une bonne pratique, mais incomplète)

Bien que l'utilisation de données anonymisées et reproductibles soit une excellente pratique de sécurité et de stabilité, elle peut devenir un frein si elle n'est pas "à la demande".

- Si le "jeu standard" est géré par une équipe externe ou s'il est rigide, il finit par limiter les tests que les développeurs peuvent imaginer.
- Pour DORA, la priorité est que la donnée serve le **flux d'automatisation** avant tout.

Action 41 - Données à la demande et automatisation - Le meilleur choix

Texte

“Utiliser des jeux de données complets pour les tests, automatisés ou non, disponibles à la demande en évitant le plus possible l'ajout de données supplémentaires.”

.**Analyse**

Des données disponibles à la demande et sans contrainte pour l'automatisation.

L'étude DORA identifie la "Gestion des données de test" comme une capacité technique clé de la **Continuous Delivery (CD)**. L'option 1 l'emporte car elle répond aux exigences des équipes les plus performantes :

- **Disponibilité à la demande :** Les équipes qui réussissent ont la capacité d'acquérir les données nécessaires à l'exécution de leurs suites de tests automatisés **sur demande**.
- **Absence de goulot d'étranglement :** Les données de test ne doivent jamais être une limite aux tests automatisés qu'une équipe peut exécuter.
- **Autonomie :** En évitant l'ajout manuel de données, l'équipe réduit le travail non planifié et le "rework", ce qui accélère le délai de livraison (Lead Time).

Action 42 - Copie de production

Texte

licensed under CC BY-NC-SA 4.0

“Utiliser pour les tests une copie des données de la production.”

.**Analyse**

La copie de production (L'anti-pattern majeur)

C'est souvent l'option choisie par défaut dans les organisations à faible performance, mais elle est vivement déconseillée pour plusieurs raisons :

- **Complexité et lourdeur :** Les données de production sont souvent trop volumineuses pour être injectées rapidement dans un pipeline de CI/CD, violant le principe de feedback rapide.
- **Sécurité et conformité :** Cela augmente les risques de fuites de données sensibles (RGPD, etc.), ce qui va à l'encontre de la capacité de "Shift Left on Security".
- **Instabilité :** Tester sur une copie de prod ne garantit pas la reproductibilité des tests, car l'état de la base change constamment.

Question bonus

La création de données de test "à la demande" est-elle un investissement trop lourd par rapport aux gains de vitesse de livraison espérés ?

Capacité 16 - Visibilité du travail dans la chaîne de valeur

Action 43 - Améliorations locales

Texte

“Les équipes IT se concentrent sur les améliorations les concernant sans avoir besoin de connaître la chaîne de valeur complète du métier jusqu'au support du produit.”

.**Analyse**

Se concentrer uniquement sur les améliorations locales (silos).

L'option 2 représente une vision fragmentée qui nuit à l'efficacité globale de l'organisation :

- **Le piège des silos :** Le mouvement DevOps vise précisément à briser les comportements où les fonctions travaillent de manière isolée sans comprendre l'ensemble de la chaîne de valeur.
- **Manque d'alignement :** Sans visibilité complète, les équipes risquent de travailler sur des tâches qui n'ont aucune valeur pour l'utilisateur final ou qui sont bloquées par d'autres étapes de la chaîne qu'elles ignorent.
- **Culture bureaucratique :** Ce type d'isolation est souvent le signe d'une culture "bureaucratique" (orientée règles) ou "pathologique" (orientée pouvoir), là où DORA prône une culture "générative" orientée vers la mission globale.

licensed under CC BY-NC-SA 4.0

Action 44 - Vision complète - Le meilleur choix

Texte

“Les équipes IT ont une bonne compréhension du flux de valeur du métier jusqu'aux utilisateurs et ont la visibilité de ce flux incluant l'état du produit et des fonctionnalités.”

.**Analyse**

La visibilité complète du métier jusqu'à l'utilisateur.

L'ouvrage *Accelerate* identifie la "Visibilité du travail dans la chaîne de valeur" comme l'une des quatre capacités clés du management de produit Lean. Cette option est la meilleure car elle répond précisément aux critères de haute performance :

- **Compréhension globale :** Les équipes doivent comprendre le flux de travail depuis l'idée métier initiale jusqu'à la livraison finale aux clients.
- **Visibilité de l'état :** Cette capacité inclut une visibilité en temps réel sur le statut des produits et des fonctionnalités au sein de ce flux.
- **Impact sur la performance :** La recherche démontre que cette visibilité a un impact positif direct sur la performance de livraison informatique.
- **Cercle vertueux :** Elle fait partie d'un cycle où l'amélioration de la livraison de logiciels permet de mieux visualiser le flux, ce qui conduit à de meilleurs résultats organisationnels.

Question bonus

Quels sont les avantages à avoir des développeurs ayant connaissance des étapes de "l'amont" (le métier) et de "l'aval" (le support/utilisateur) ?

Capacité 17 - Retours du client

Action 46 - Collecte régulière - Le meilleur choix

Texte

“La collecte des retours utilisateurs est effectuée de manière régulière et les équipes ont la possibilité d'effectuer des corrections en conséquence sur la conception ou les spécifications.”

.**Analyse**

Collecte régulière et autonomie de correction sur la conception.

Le "recueil et l'implémentation des retours clients" est une capacité clé du management de produit Lean. Cette option est la meilleure car elle remplit des critères essentiels :

licensed under CC BY-NC-SA 4.0

- **L'intégration continue du feedback** : Les organisations les plus performantes cherchent activement et régulièrement les retours des clients et les intègrent directement dans la conception de leurs produits.
- **L'autorité de l'équipe (Team Experimentation)** : Un facteur déterminant de la performance est que les équipes de développement aient l'autorité de créer et de modifier les spécifications au cours du processus de développement sans nécessiter d'approbation externe.
- **Le cycle de validation** : Travailler par petits lots permet d'obtenir ces retours rapidement pour corriger le tir (course correct) avant qu'il ne soit trop tard ou trop coûteux.

Action 47 - Collecte après déploiement

Texte

“La collecte des retours utilisateurs est effectuée après le déploiement en production à des fins d'améliorations.”

.**Analyse**

Collecte uniquement après le déploiement.

Bien que recueillir des retours après la mise en production soit nécessaire, s'en contenter comme seul mécanisme d'amélioration présente des risques identifiés :

- **Le goulot d'étranglement des spécifications** : Si les équipes sont obligées de suivre des spécifications figées créées par d'autres équipes sans pouvoir les changer, leur capacité à innover est fortement inhibée.
- **Feedback trop tardif** : Attendre la fin du déploiement pour découvrir qu'une fonctionnalité ne répond pas au besoin est une forme de gaspillage (Muda). La science DORA montre que l'approche expérimentale doit être corrélée aux pratiques techniques de livraison continue pour être efficace.
- **Absence de cycle vertueux** : La haute performance repose sur un "cercle vertueux" où la vitesse de livraison permet de valider des idées plus souvent, augmentant ainsi la satisfaction au travail et la performance organisationnelle.

Question bonus

Est-ce difficile d'obtenir l'autorisation de changer les spécifications en cours de route, ou cela fait-il déjà partie de votre culture agile ?

Capacité 18 - Management visuel

Action 54 - Suivi d’activité

licensed under CC BY-NC-SA 4.0

Texte

“Mise en place de tableaux Kanban partagés entre équipes IT afin de suivre l'évolution de l'activité.”

.**Analyse**

Le piège du suivi d'activité pur.

Cette option est souvent ce que l'on appelle une "mesure de sortie" (output) plutôt qu'une "mesure de résultat" (outcome).

- **Vision fragmentée :** Se concentrer uniquement sur l'évolution de l'activité IT (les tâches Kanban) ne dit rien sur la qualité du produit ou sur la satisfaction de l'utilisateur final.
- **Effet limité :** La recherche indique que le suivi de l'activité seul ne prédit pas fortement la performance de livraison si elle n'est pas corrélée à la qualité et au feedback de la production.
- **Risque de "Vanity Metrics" :** Suivre l'activité peut devenir une mesure de vanité si cela ne permet pas de prendre des décisions commerciales quotidiennes basées sur des données réelles de performance.

Action 55 - Tableaux de bord complets orientés système - Le meilleur choix

Texte

“Mise en place de tableaux de bord visuels ouverts à tous incluant le suivi d'activités, le monitoring du pipeline de déploiement, les informations principales sur télémétrie Production.”

.**Analyse**

Des tableaux de bord complets, transparents et orientés "système".

Cette approche est un moteur de performance car elle ne se contente pas de montrer "qui travaille sur quoi", mais elle expose l'état de santé global de la livraison de valeur :

- **Alignement avec les objectifs opérationnels :** Les recherches montrent que les affichages visuels doivent inclure des indicateurs clés de qualité et de productivité, mis à la disposition des ingénieurs comme des leaders.
- **La puissance de la boucle de feedback :** L'une des découvertes majeures est que les outils de management visuel (comme les limites de WIP) n'ont un effet positif fort que lorsqu'ils sont combinés à une boucle de feedback provenant des outils de monitoring de production.

licensed under CC BY-NC-SA 4.0

- **Accessibilité et Transparence :** Les hauts performeurs utilisent des affichages visuels (tableaux de bord ou sites internes) pour partager l'information sur l'état actuel du travail et les métriques de qualité avec l'ensemble de l'organisation.
- **Culture Générative :** En ouvrant ces données à tous, on favorise une culture où l'information est activement recherchée et où les échecs sont traités comme des opportunités d'amélioration du système global.

Question bonus

Qu’est-ce qu’un bon management visuel ? Etes-vous à l'aise avec l'idée de rendre les "erreurs de production" visibles à tous, ou y a-t-il une crainte liée à la culture du blâme ?

Le management visuel n'est pas une fin en soi, mais un outil de communication de haute qualité :

1. **Visibilité du flux :** Un bon tableau de bord doit permettre aux équipes de comprendre et de visualiser le flux de travail depuis le métier jusqu'au client.
2. **Impact sur le moral :** Ces pratiques de Lean Management ne font pas que booster la vitesse ; elles réduisent statistiquement le burnout des équipes en rendant le travail plus clair et les obstacles plus visibles pour être supprimés.
3. **Engagement :** Lorsque les employés ont les outils et les ressources (y compris l'information) pour faire leur travail, leur satisfaction et leur identification à l'entreprise augmentent.

Capacité 25 - Les métriques comme objectif

Aucune des actions proposées n’est correcte

Ceci était un piège: Mettre en visibilité les valeurs des métriques DORA est une façon de s'améliorer mais l'amélioration des métriques seules n'est pas l'objectif (cf Loi de Goodhart: " Lorsqu'une mesure devient un objectif, elle cesse d'être une bonne mesure").

L’objectif au travers de cette fausse capacité est de souligner l’importance de ne pas effectuer un pilotage par les métriques mais plutôt de monitorer les résultats au travers d’actions, résultats qui ne sont pas linéaires.

Attention : Les OKR sont une bonne approche, le problème vient de l’objectif “métriques” en lui même.

licensed under CC BY-NC-SA 4.0